

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 7

End-to-End Study of Autoencoders, Convolutional Autoencoders, Denoising Autoencoders and Variational Autoencoders

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

The objective of this experiment is to develop an end-to-end understanding of autoencoders and their variants for image representation, reconstruction, denoising and generative modeling.

The experiment begins with a fully connected autoencoder, progresses to a Convolutional Autoencoder (CAE), introduces image corruption for denoising, and finally develops a Variational Autoencoder (VAE). Students will visualize the learned latent representation, compare reconstruction quality using quantitative and qualitative measures, and explore the generative capability of the VAE.

The same image dataset and a controlled experimental protocol should be used throughout the experiment wherever possible.

2. Learning Outcomes

After completing this experiment, students will be able to:

- explain the encoder–latent–decoder structure of an autoencoder;
- prepare image data for reconstruction-based learning;
- implement a fully connected autoencoder;
- implement a Convolutional Autoencoder for image reconstruction;
- introduce controlled noise and build a denoising autoencoder;
- evaluate reconstruction using MSE, MAE and SSIM;
- visualize and interpret latent representations;
- explain the probabilistic latent representation of a VAE;
- implement the reparameterization trick;
- calculate and interpret reconstruction and KL-divergence losses;
- generate new images by sampling from a learned latent space; and
- compare deterministic and variational latent representations.

3. Dataset and Experimental Setup

Dataset: MNIST Handwritten Digit Dataset

MNIST contains grayscale images of handwritten digits from 0 to 9. Each image has spatial dimensions

$$28 \times 28 \times 1.$$

The dataset is particularly suitable for this experiment because the images are small, the classes have visually meaningful structure, and the same image format can be directly supplied to fully connected, convolutional and variational autoencoders.

The dataset can be loaded directly using Keras/TensorFlow:

https://www.tensorflow.org/api_docs/python/tf/keras/datasets/mnist

Recommended laboratory subset:

Use 10,000 training images and 2,000 test images for the main laboratory execution. The full MNIST dataset may be used if computational resources permit.

The labels are not required as reconstruction targets. The original image is the target:

$$x \rightarrow \text{Encoder} \rightarrow z \rightarrow \text{Decoder} \rightarrow \hat{x}$$

where x is the input image and $\hat{x}$ is its reconstruction.

4. Expected Input and Output

Expected Input

Each input image has shape

$$28 \times 28 \times 1.$$

For a batch of 32 images:

$$X \in \mathbb{R}^{32 \times 28 \times 28 \times 1}.$$

Pixel values should be converted from

$$[0, 255] \rightarrow [0, 1].$$

For a fully connected autoencoder, the image may be flattened:

$$28 \times 28 = 784.$$

Therefore,

$$x \in \mathbb{R}^{784}.$$

For a convolutional autoencoder, retain the spatial structure:

$$x \in \mathbb{R}^{28 \times 28 \times 1}.$$

Expected Output

The primary output is a reconstructed image:

$$\hat{x} \in \mathbb{R}^{28 \times 28 \times 1}.$$

Students must display:

Original image
Reconstructed image
Reconstruction error

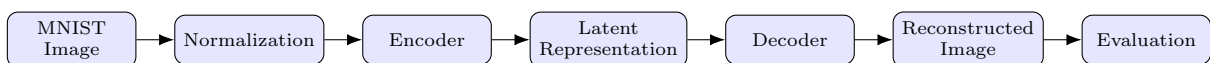
For the denoising model, display:

Original clean image
Noisy image
Denoised reconstruction

For the VAE, display newly generated samples obtained by sampling from the latent distribution.

5. Overall Experimental Pipeline

The complete experiment follows:



The experiment consists of four related studies:

1. Fully Connected Autoencoder
2. Convolutional Autoencoder
3. Denoising Convolutional Autoencoder
4. Variational Autoencoder

6. Exploring Autoencoders

An autoencoder learns to reproduce its input at the output.

The encoder maps the input to a lower-dimensional representation:

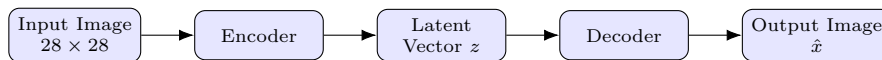
$$z = f_{\theta}(x),$$

and the decoder reconstructs the input:

$$\hat{x} = g_{\phi}(z).$$

The training objective is to minimize reconstruction error:

$$\mathcal{L}_{rec} = \frac{1}{N} \sum_{i=1}^N \|x_i - \hat{x}_i\|^2.$$



The latent vector is a compressed representation of the input.

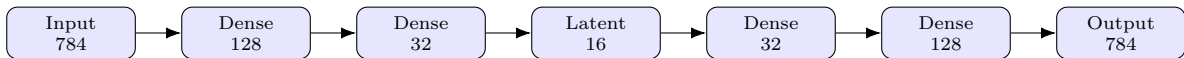
7. Fully Connected Autoencoder

Flatten each image from

$$28 \times 28 = 784$$

to a vector of length 784.

Use the following architecture:



Use ReLU activation in hidden layers and sigmoid activation in the output layer.

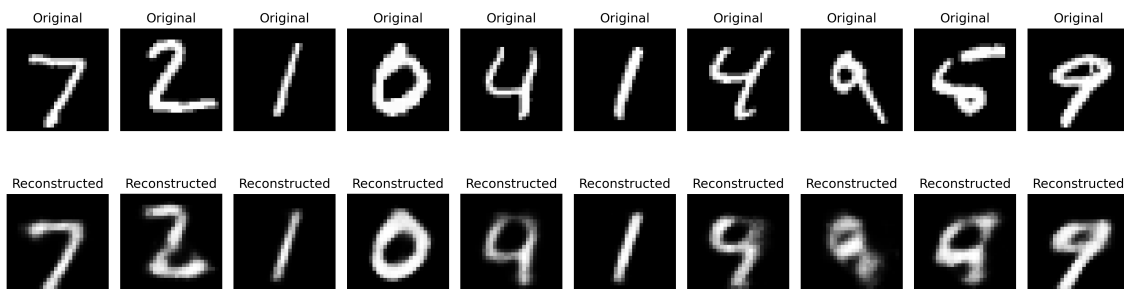
Suggested configuration:

Parameter	Value
Latent dimension	16
Optimizer	Adam
Learning rate	10^{-3}
Batch size	128
Epochs	20
Loss	Binary Cross-Entropy

8. Required Analysis of the Basic Autoencoder

Plot 1: Original versus Reconstructed Images

The original test images are compared with their corresponding reconstructions produced by the fully connected autoencoder.

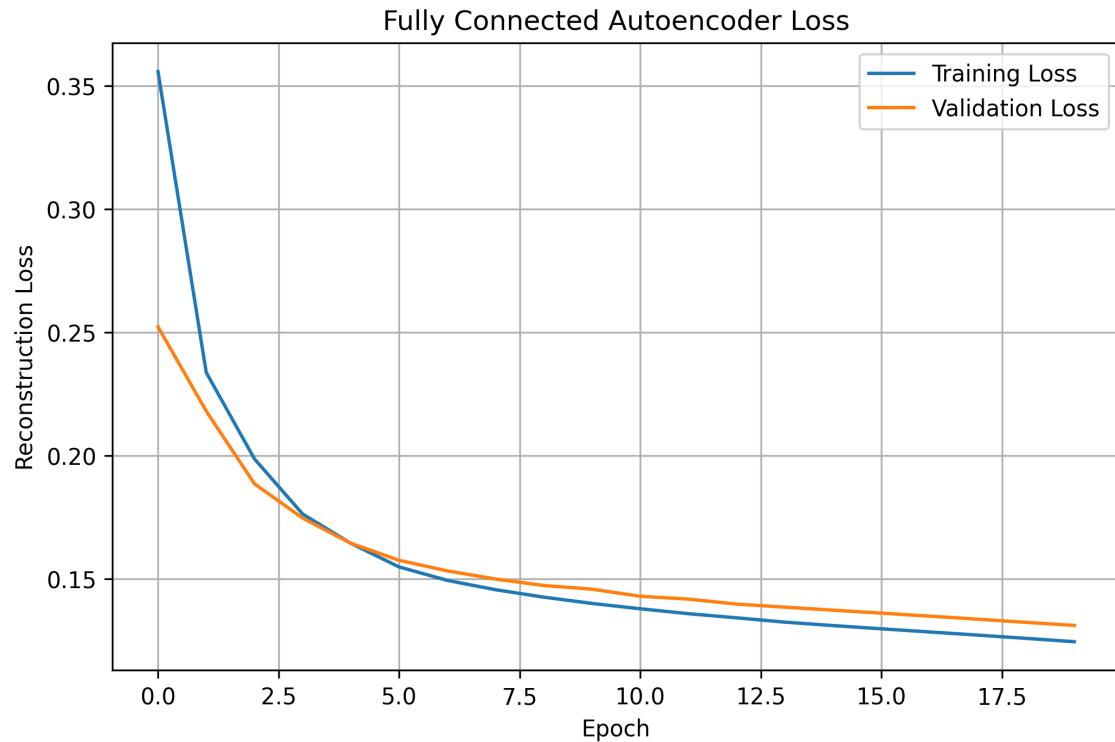


Inference:

The fully connected autoencoder can still recreate the overall layout of the handwritten digits. In many of the outputs, the main strokes look the same as in the input images. That said, the reconstructions do not keep every small feature. Some parts look a bit soft or smeared when you compare them to the originals. Digits that are simple and easy to read tend to match better. When the digit has lots of strokes or a confusing shape, the mismatch becomes bigger. One reason is the way the model uses a 16 dimensional code. It has to squeeze the original 784 dimensional image into this much smaller vector.

Plot 2: Training and Validation Reconstruction Loss

The training and validation reconstruction losses were monitored over 20 epochs.



Inference:

Both the training and validation reconstruction losses drop steadily across all 20 epochs. This suggests the fully connected autoencoder is learning and reaching a stable state. The validation curve goes down in step with the training curve. It also does not stay higher for any long stretch while the training loss keeps falling. Because of that pattern, there is no strong sign of overfitting in this run. At the end, the training loss is about 0.1244. The validation loss is about 0.1311. The two values are close, so the reconstruction quality looks similar for both sets.

Overall Inference:

A simple fully connected autoencoder can learn a smaller form of MNIST digits. It also rebuilds the images and keeps the main visual layout. In the training plot, the loss goes down in a steady way. The output images look close to the digits in the input. Still, some small texture and subtle parts do not come back. This happens because the hidden code has too few dimensions.

9. Reconstruction Metrics

Students must calculate the following metrics on the test set.

Mean Squared Error

$$MSE = \frac{1}{N} \sum_{i=1}^N (x_i - \hat{x}_i)^2.$$

Mean Absolute Error

$$MAE = \frac{1}{N} \sum_{i=1}^N |x_i - \hat{x}_i|.$$

Structural Similarity

Calculate SSIM between the original and reconstructed images.

The implementation may use:

<https://scikit-image.org/docs/stable/api/skimetrics.html>

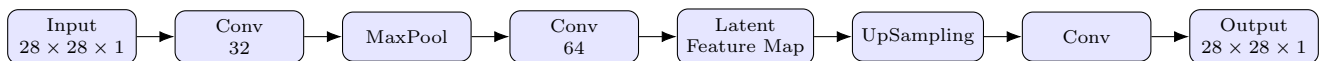
Report:

Metric	Value
Test MSE	0.0223
Test MAE	0.0590
Mean SSIM	0.7326

10. Convolutional Autoencoder

A Convolutional Autoencoder preserves spatial structure and uses convolutional operations in the encoder and upsampling/transposed convolution operations in the decoder.

The conceptual architecture is:



A possible implementation is:

```
Input(28,28,1)
Conv2D(32, 3, activation='relu', padding='same')
MaxPooling2D(2, padding='same')
Conv2D(64, 3, activation='relu', padding='same')
MaxPooling2D(2, padding='same')
Conv2D(64, 3, activation='relu', padding='same')
UpSampling2D(2)
Conv2D(32, 3, activation='relu', padding='same')
UpSampling2D(2)
Conv2D(1, 3, activation='sigmoid', padding='same')
```

The exact implementation may use Conv2DTranspose instead of UpSampling2D.

11. Comparing Fully Connected and Convolutional Autoencoders

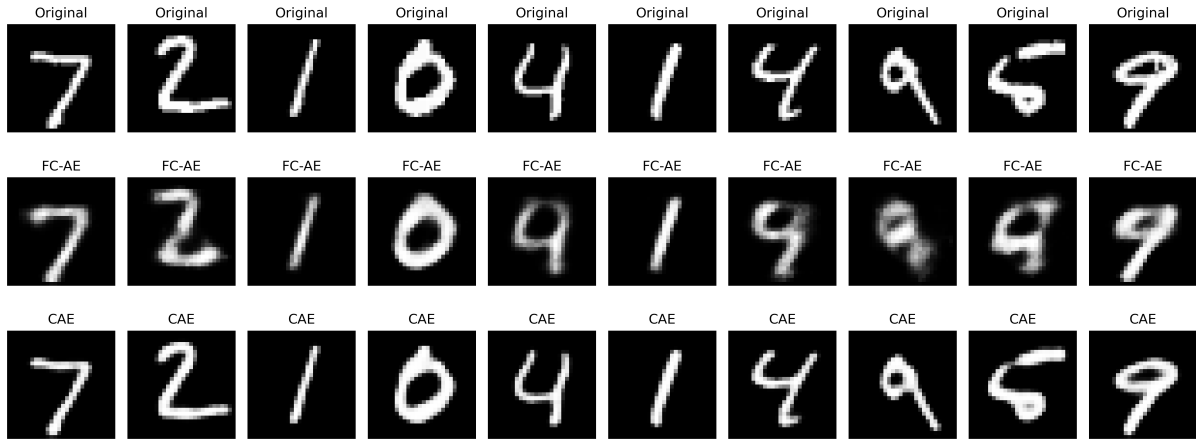
Train both models using the same training and test images.

Model	MSE	MAE	SSIM	Parameters
Fully Connected AE	0.0223	0.0590	0.7326	211040
Convolutional AE	0.0025	0.0150	0.9739	74497

Plot 3: Reconstruction Comparison

Display:

Original → FC-AE Reconstruction → CAE Reconstruction.



Required inference:

The convolutional autoencoder gives a smaller reconstruction loss than the fully connected one. It also matches the input structure more closely. For the CAE, the MSE is 0.002581, the MAE is 0.015000, and the SSIM reaches 0.973997. For the fully connected autoencoder, the values are MSE 0.022371, MAE 0.059099, and SSIM 0.732669.

Part of the reason is that convolutional layers keep nearby pixel information. They can learn local patterns from images. In handwritten digits, close pixels often belong to the same strokes and shapes. Convolutions pick up those local details better than dense layers do. The CAE reaches these results while using fewer parameters too. It uses 74,497 parameters, while the fully connected model uses 211,040.

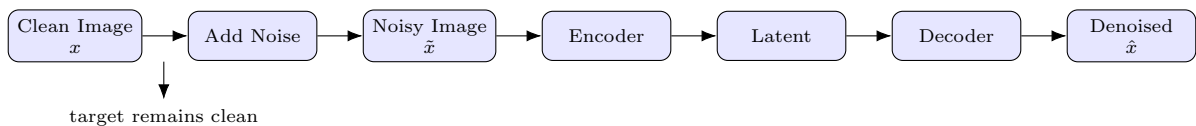
12. Denoising Autoencoder

A denoising autoencoder receives a corrupted image but is trained to reconstruct the original clean image.

The training mapping is:

$$\tilde{x} \rightarrow \text{Encoder} \rightarrow z \rightarrow \text{Decoder} \rightarrow \hat{x}$$

where $\tilde{x}$ is a noisy version of the clean image x .



13. Adding Image Noise

Use at least two corruption types.

Gaussian Noise

$$\tilde{x} = x + n, \quad n \sim \mathcal{N}(0, \sigma^2).$$

Suggested standard deviations:

$$\sigma \in \{0.1, 0.2, 0.3\}.$$

Clip the resulting image to the range $[0, 1]$.

Salt-and-Pepper Noise

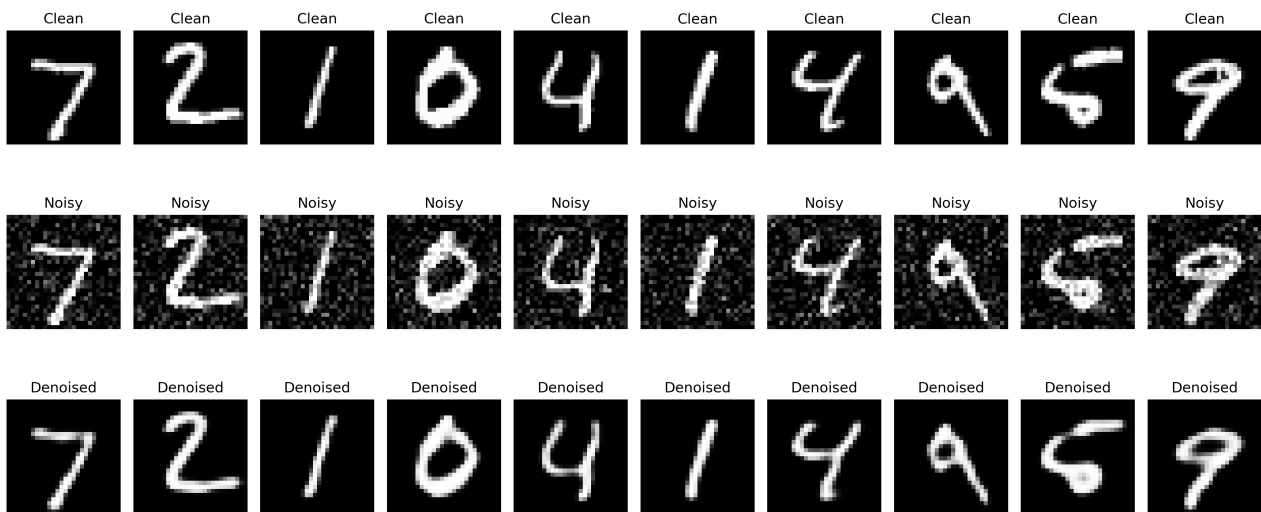
Randomly replace a small proportion of pixels with values close to 0 or 1.

Suggested corruption levels:

$$p \in \{0.05, 0.10, 0.20\}.$$

14. Required Denoising Plots

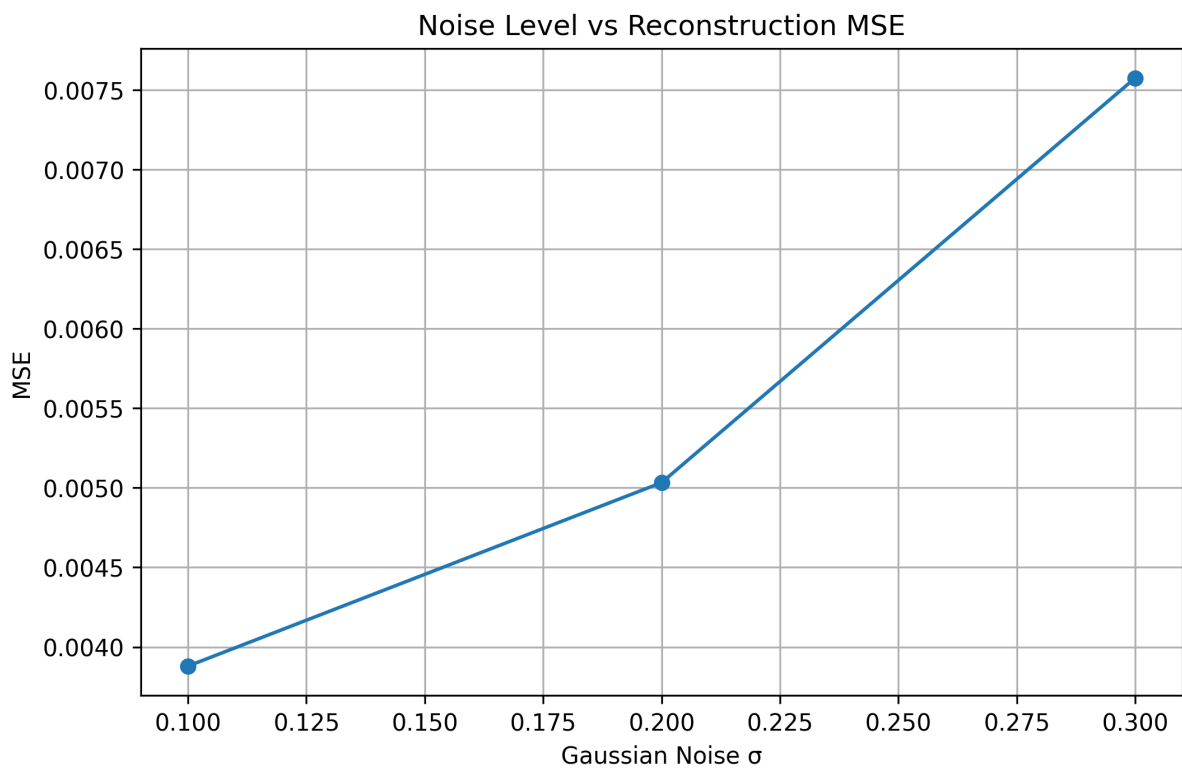
Plot 4: Clean vs. Noisy vs. Denoised

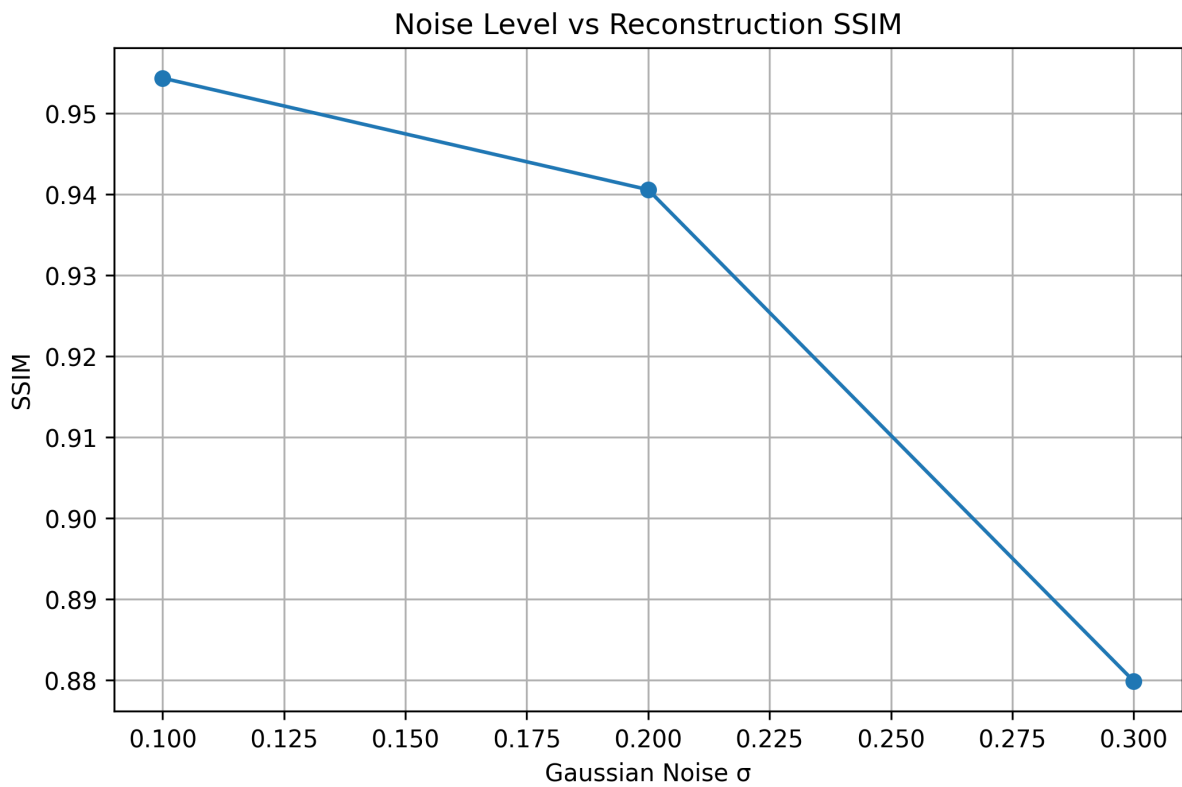
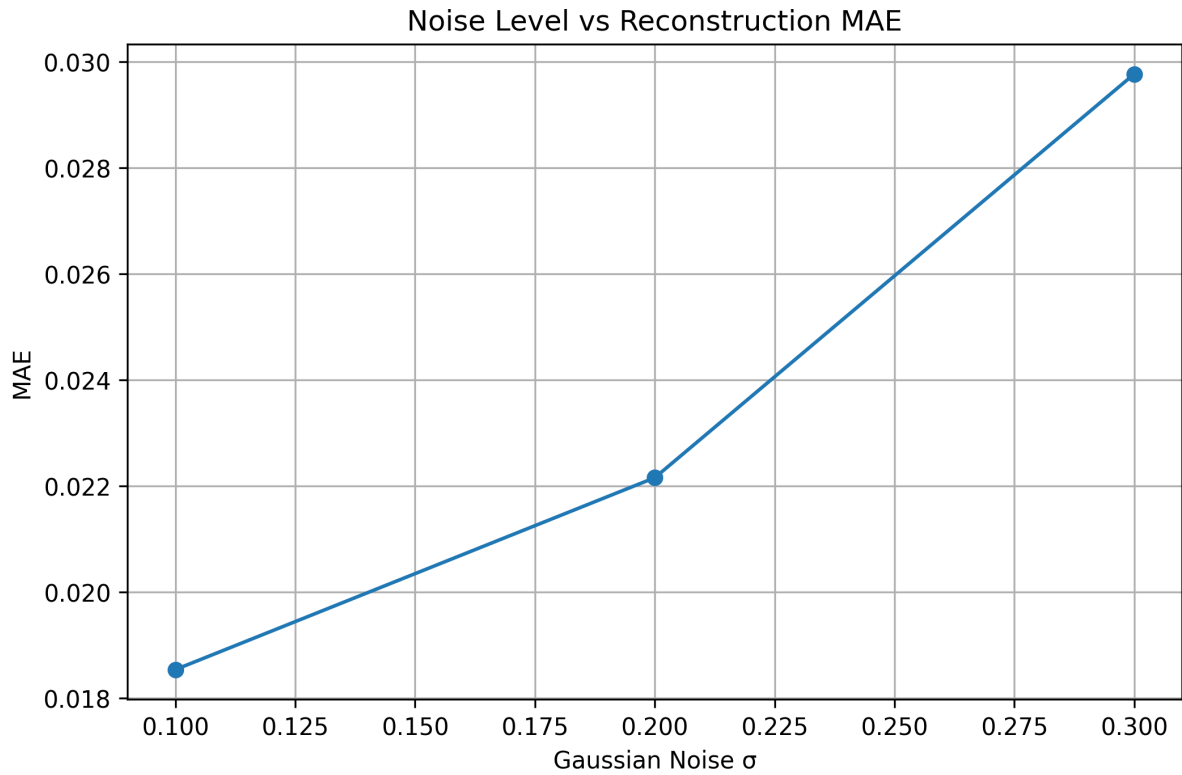


Plot 5: Noise Level vs. Reconstruction Quality

Report:

Noise Level	MSE	MAE	SSIM
0.1	0.0038	0.0185	0.9543
0.2	0.0050	0.0221	0.9405
0.3	0.0075	0.0297	0.8799





Required inference:

When the noise gets higher, the reconstruction gets worse. I changed sigma from 0.1 to 0.3. In that span, MSE rises from 0.003880 to 0.007575. MAE also goes up, from 0.018540 to 0.029765.

At the same time, SSIM drops. It falls from 0.954337 to 0.879907. This suggests that the noisy images lose more of their shared structure.

Even with the added corruption, the denoising autoencoder still keeps the main shape of the handwritten digits. It can still restore a lot of the original image layout from the damaged input.

15. Variational Autoencoder

A conventional autoencoder maps an input to a deterministic latent vector:

$$z = f(x).$$

A VAE instead learns a probability distribution:

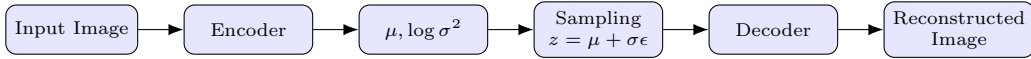
$$q_\phi(z|x) = \mathcal{N}(\mu(x), \text{diag}(\sigma^2(x))).$$

The encoder produces:

$$\mu, \quad \log \sigma^2.$$

A latent vector is sampled using the reparameterization trick:

$$z = \mu + \sigma \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I).$$



16. VAE Loss Function

The VAE objective consists of two components.

Reconstruction Loss

$$\mathcal{L}_{rec} = -\mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)].$$

For normalized MNIST images, binary cross-entropy can be used.

KL Divergence

$$D_{KL}(q_\phi(z|x) \parallel p(z)) = -\frac{1}{2} \sum_j (1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2).$$

The total loss is

$$\boxed{\mathcal{L}_{VAE} = \mathcal{L}_{rec} + \mathcal{L}_{KL}}$$

where the prior is usually

$$p(z) = \mathcal{N}(0, I).$$

17. VAE Implementation

Use a small latent space so that it can be visualized.

Suggested latent dimension:

$$z \in \mathbb{R}^2.$$

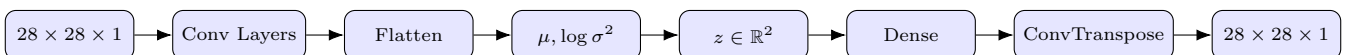
The encoder should produce:

$$\mu \in \mathbb{R}^2, \quad \log \sigma^2 \in \mathbb{R}^2.$$

The decoder maps

$$z \rightarrow 28 \times 28 \times 1.$$

A compact architecture is:

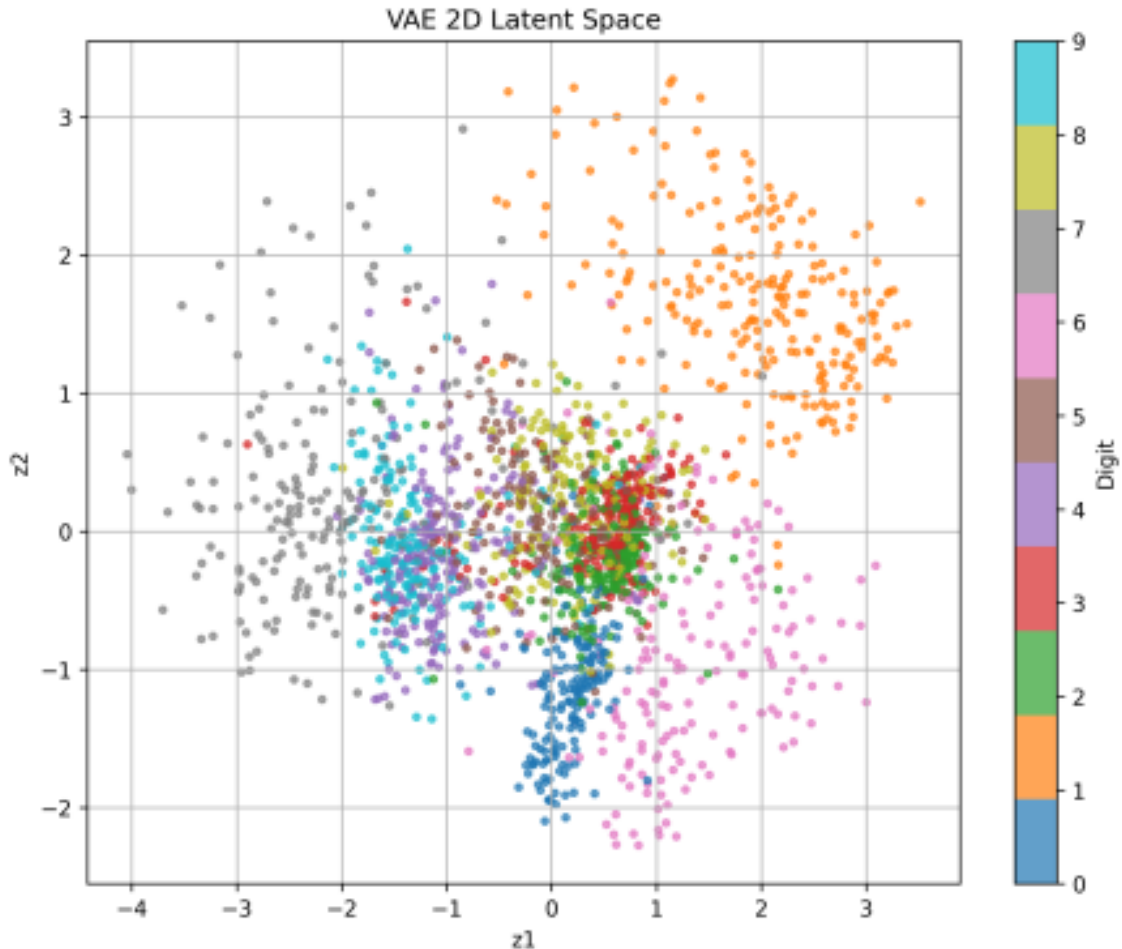


18. Exploring the VAE Latent Space

Since the latent dimension is two, students can visualize the encoded test images in a two-dimensional plane.

Plot 6: VAE Latent Space

- X-axis: z_1
- Y-axis: z_2
- Color/marker category: digit label



The labels are used only for visualization; they are not supplied to the VAE during training.

Required inference:

The picture in two dimensions helps show where the test images land in the latent space. Each point stands for a sample, placed using its learned latent features.

If you look at samples that share the same digit, you can see whether they end up near one another. When images of the same digit cluster in similar spots, it suggests the model is grouping them in the latent space.

You can also compare different digits. If the areas for two digit groups overlap a lot, then those classes are not well separated in the latent space. If there is less overlap, the digit classes are more clearly separated in how the VAE stores them.

Overall, the plot gives a way to view the structure the model learned in its probabilistic latent representation.

19. Generating New Images with the VAE

Sample points from

$$z \sim \mathcal{N}(0, I).$$

Pass the sampled latent vectors through the decoder.

Plot 7: Randomly Generated Images

VAE Generated Images



The generated samples were examined based on the following characteristics:

- resemblance to handwritten MNIST digits;
- recognizability of the digit shapes;
- variation among the generated samples; and
- presence of ambiguous or unrealistic samples.

Inference:

The VAE can make new images by drawing samples from the latent space it learned. This is not the same as reusing the test pictures. When you look at the output, you can judge how good it is by checking if the digit parts still look like digits. You can also compare results to see if the samples differ in a sensible way.

In some cases, the digits do not look sharp. The shapes can be unclear or hard to tell apart. This happens because the latent model and the reconstruction ability of the trained VAE have limits.

20. Latent Space Interpolation

Select two latent vectors:

$$z_A, \quad z_B.$$

Construct

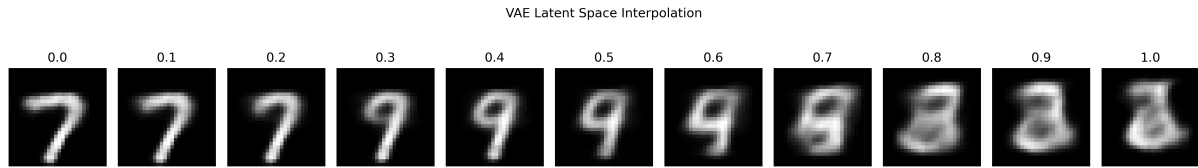
$$z(\alpha) = (1 - \alpha)z_A + \alpha z_B, \quad \alpha \in [0, 1].$$

Use values such as:

$$\alpha = 0, 0.1, 0.2, \dots, 1.$$

Decode every interpolated vector.

Plot 8: Latent Space Interpolation



Required inference:

The interpolation shows what the decoder does as the latent values shift little by little. When the output images look steady and move smoothly along the sequence, it suggests that close latent points map to image changes that also happen gradually. If the images jump suddenly, or if the middle frames look hard to interpret, that usually means the latent space has areas where smooth change is not well supported.

21. Reconstruction Comparison Across Models

Compare the three reconstruction-based models:

Model	MSE	MAE	SSIM	Parameters
Fully Connected AE	0.0223	0.0590	0.7326	211,040
Convolutional AE	0.0025	0.0150	0.9739	74,497
Denoising CAE	0.0049	0.0220	0.9413	74,497

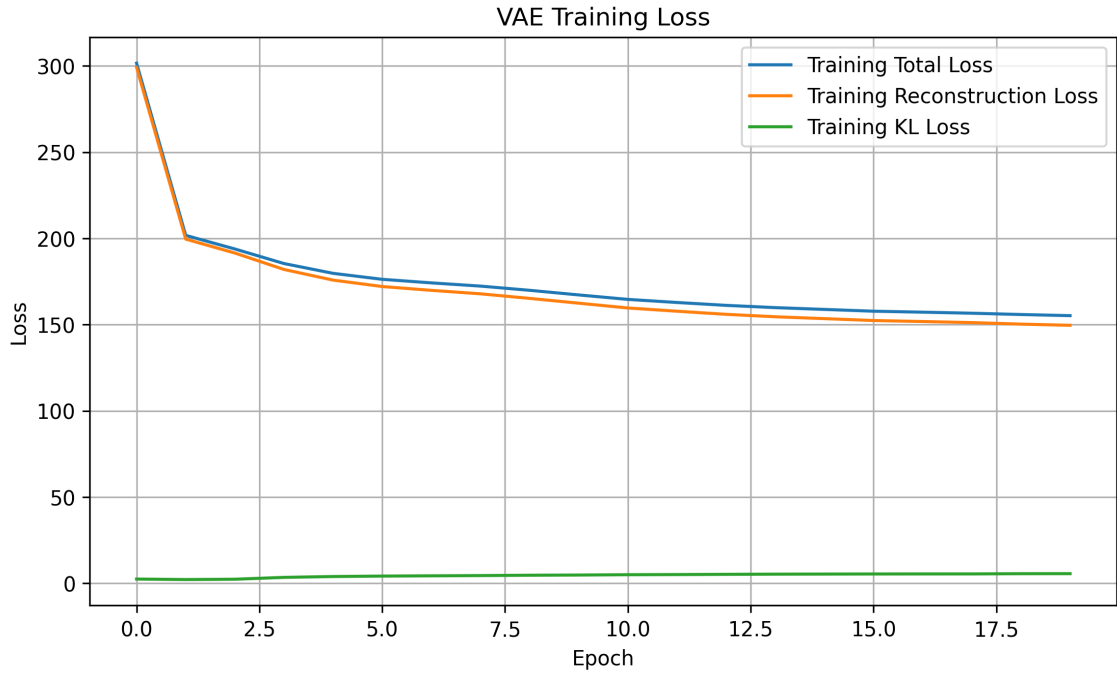
For the VAE, report its reconstruction loss separately because its total objective also contains the KL-divergence term.

VAE Metric	Value
Reconstruction Loss	149.5990
KL Loss	5.5968
Total Loss	155.1958
Test MSE	0.0460
Test MAE	0.1059
Mean SSIM	0.4779

22. Required Plots

The plots already presented in the previous sections are not repeated here. The remaining required plot is the VAE training and validation reconstruction loss.

Plot 9: VAE Training and Validation Reconstruction Loss



Inference:

The figure presents the reconstruction loss for a VAE in both training and validation. During training, the reconstruction loss goes down. This happens as the model learns to map the input images into a latent space. Then it rebuilds the images using the decoder.

The validation line shows what happens on data the model has not seen yet. It tells you how the reconstruction quality shifts as training continues.

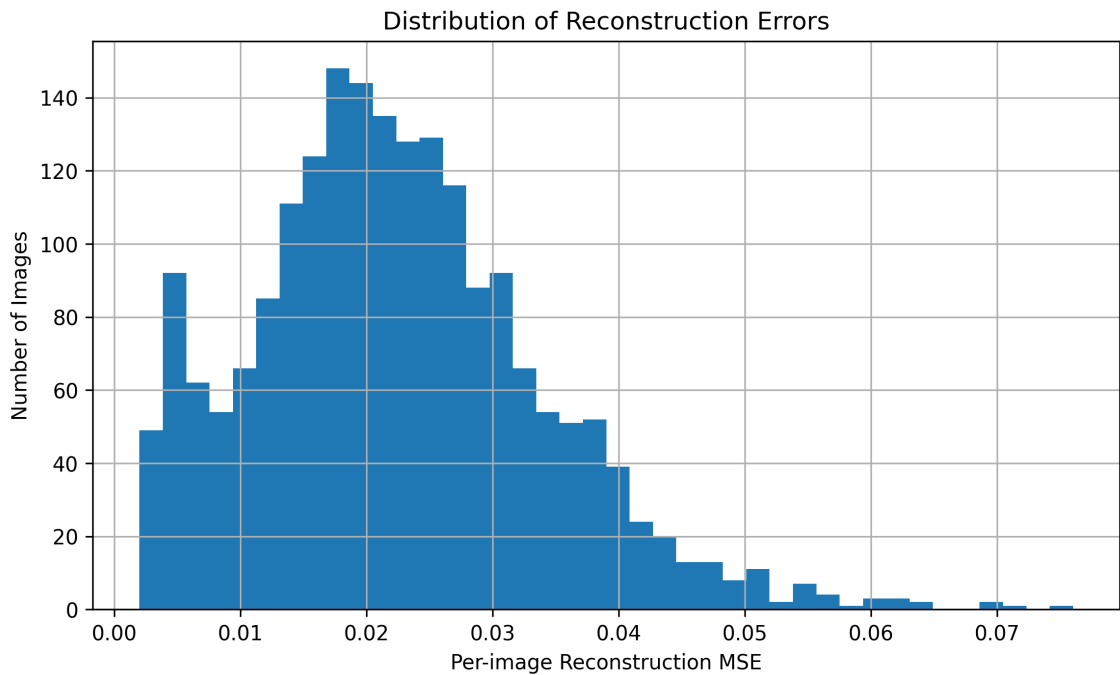
23. Reconstruction Error Distribution

Calculate the per-image reconstruction error:

$$e_i = \frac{1}{D} \sum_{j=1}^D (x_{ij} - \hat{x}_{ij})^2.$$

Plot 10: Distribution of Reconstruction Errors

Plot a histogram of e_i over the test set.



Inference:

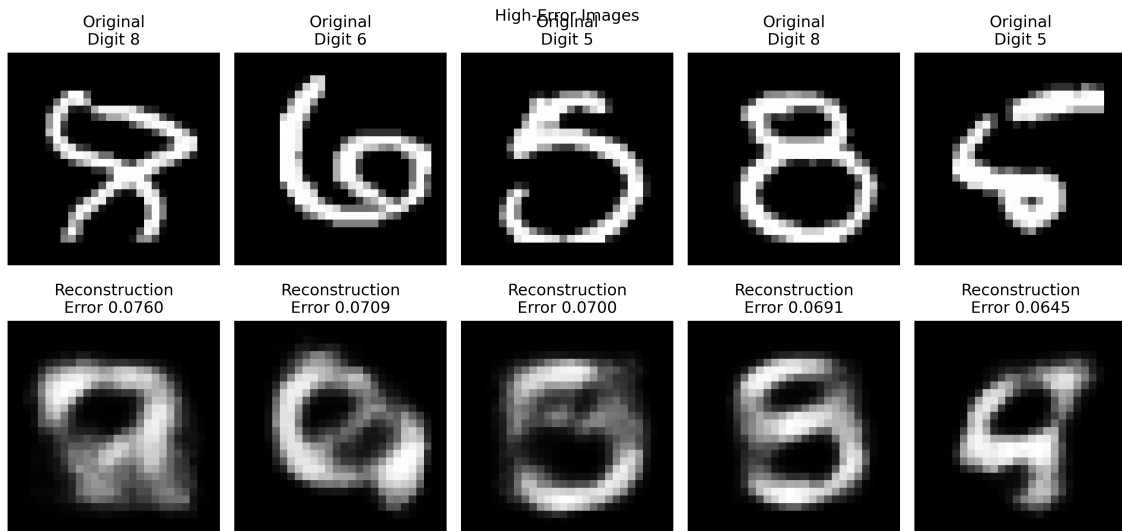
The reconstruction error curve shows how well the autoencoder rebuilds each test image. For most images, the error stays fairly small. A few images stand out with much larger values. The average error is 0.022371154. The largest one is 0.07601914.

Those higher-error cases likely mean the model did not keep the pixel details as well as it did for the rest. You can look into them more in the next part, where the high-error images are reviewed.

24. Required High-Error Image Analysis

Select the five test images having the largest reconstruction errors.

Display:



Inference:

The samples with higher error are test images that the autoencoder has trouble matching well. In these cases, the output can differ from the original. This often happens when the digit has harder parts to capture, like thicker or more tangled lines, odd pen shapes, parts that sit on top of each other, or small strokes that the latent form does not keep.

Across the test set, the error level is not the same for every image. Some images, especially those with busy or less common visual layouts, tend to lead to higher reconstruction error. Simpler cases usually result in lower error.

25. Consolidated Results

Model	MSE	MAE	SSIM	Parameters	Time
FC Autoencoder	0.0223	0.0590	0.7326	211,040	22.24
Conv. Autoencoder	0.0025	0.0150	0.9739	74,497	18.24
Denoising CAE	0.0049	0.0220	0.9413	74,497	17.70
VAE	0.0460	0.1059	0.4779	485,957	21.74

Inference:

The combined results list how well each of the four models reconstructs images. The convolutional autoencoder gets good scores on MSE and MAE and also keeps SSIM high. The denoising CAE still shows strong SSIM even though it is trying to recover clean images from noisy ones. The fully connected autoencoder uses more parameters than the convolutional version, but its reconstruction errors come out higher.

The VAE stands out because it has the most parameters of the four. It also uses a probabilistic latent space. In the report, the reconstruction numbers are shown apart from the KL-divergence part. This is because the training loss has both a reconstruction term and a KL-divergence term.

26. Required Inferences

1. Characteristics of the VAE Latent Distribution

Unlike the deterministic latent representation of a conventional autoencoder, the VAE represents each input using a probability distribution parameterized by μ and $\log \sigma^2$. The reparameterization

$$z = \mu + \sigma \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$$

allows latent samples to be obtained while maintaining a differentiable training process. The KL-divergence term encourages this learned distribution to remain close to the standard normal prior.

2. Structure of the Two-Dimensional VAE Latent Space

The model maps the test samples into a 2D latent space. From that space, you can view the encoded points directly. The latent-space plot also tells you how the digit samples sit in the learned representation. When the points overlap, it means different digit patterns end up in nearby latent areas. When the points stay apart, it suggests the digits have more separate representations.

3. Quality and Diversity of Generated Samples

A VAE can make fresh images. It does this by taking random latent vectors drawn from a standard normal distribution. Those vectors are then fed into the decoder to get an output image.

You will notice the results differ from one sample to the next. That happens because each latent vector maps to a different decoded image.

Still, not every image looks sharp. A few outputs can feel vague or hard to pin down. This is likely due to limits in how the model learned its latent space.

4. Smoothness of Latent-Space Interpolation

Latent-space interpolation shows what the decoder does when the latent variables move smoothly. When you shift between two decoded results step by step, the images change in a steady way. That steady shift suggests the latent space learned by the model allows gradual movement from one image representation to another.

5. Deterministic versus Probabilistic Latent Representations

An ordinary autoencoder turns the input into one fixed latent vector. A VAE instead learns a distribution over the latent space. Because of that, the VAE can draw several different latent vectors. That extra randomness helps it make images, not only rebuild the input.

27. Additional Latent-Dimension Study

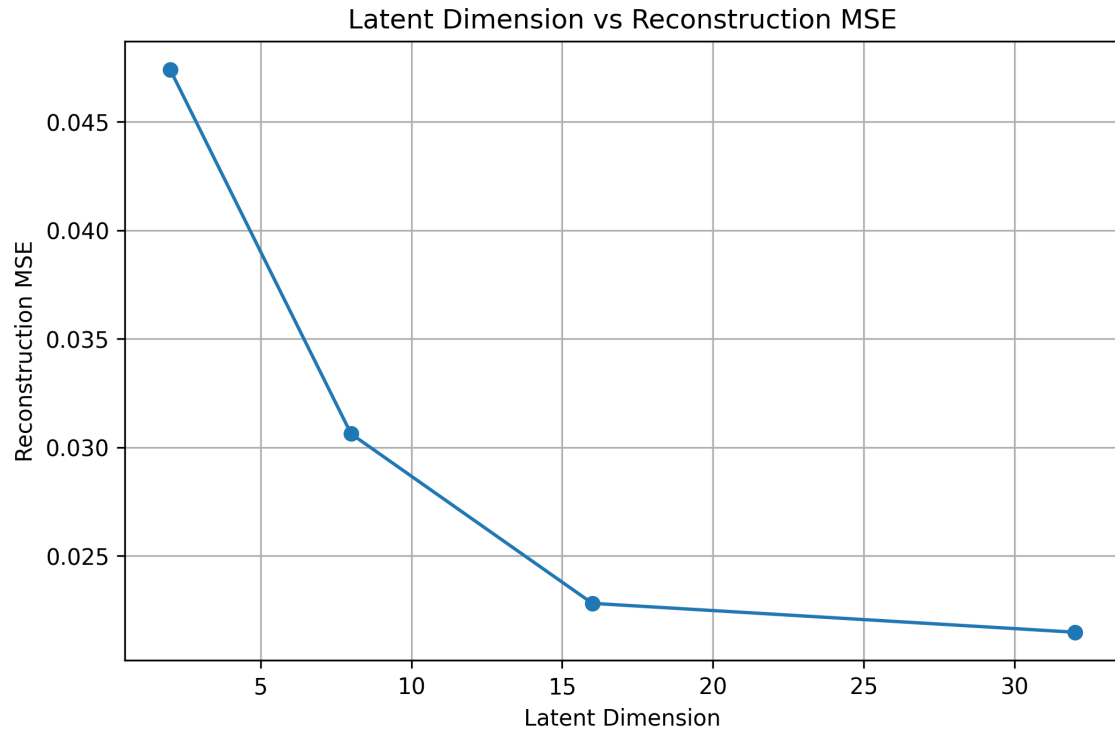
Repeat the basic autoencoder using:

$$d_z \in \{2, 8, 16, 32\}.$$

Report:

Latent Dimension	MSE	SSIM	Parameters
2	0.0474	0.4386	210130
8	0.0306	0.6437	210520
16	0.0228	0.7275	211040
32	0.0214	0.7438	212080

Plot 10: Latent Dimension vs. Reconstruction MSE



Inference:

When the latent size goes from 2 up to 32, the reconstruction MSE drops from 0.047406 to 0.021482. At the same time, SSIM rises from 0.438639 to 0.743899. Overall, this suggests that a bigger latent code keeps more details from the original image, so the reconstruction looks better.

With latent dimension 2, the model compresses the most. But it also gives the largest reconstruction error. As you move to larger latent dimensions, the model can store more image features. That is why the reconstruction error keeps going down, and SSIM goes up. The cost is a small bump in trainable weights, from 210130 at dimension 2 to 212080 at dimension 32.

So the main takeaway is a trade-off. A smaller latent dimension gives stronger compression but throws away more information. A larger latent dimension keeps more information and gives better reconstructions.

28. Discussion Questions

1. What is an autoencoder?

An autoencoder is a neural network that learns to reproduce its input at the output. It consists of an encoder that maps the input to a latent representation and a decoder that reconstructs the input from that representation.

2. What is the purpose of the encoder?

The encoder transforms the input image into a lower-dimensional latent representation containing the important information required for reconstruction.

3. What is the purpose of the decoder?

The decoder takes the latent representation produced by the encoder and reconstructs the original input image.

4. What is a latent representation?

A latent representation is a compressed representation of the input learned by the encoder. It contains the important features required by the decoder to reconstruct the input.

5. Why is the latent representation usually smaller than the input?

A smaller latent representation forces the network to learn the most important features of the input rather than simply copying the input directly.

6. What happens when the latent dimension is too small?

When the latent dimension is too small, the model has insufficient capacity to preserve all important information. This generally results in larger reconstruction errors and lower reconstruction quality.

7. What happens when the latent dimension is increased?

Increasing the latent dimension provides more capacity to represent the input information. In the latent-dimension experiment, increasing the dimension from 2 to 32 reduced the reconstruction MSE and increased the SSIM.

8. Why can a convolutional autoencoder preserve spatial information more effectively?

Convolutional layers operate on local regions of the image and preserve spatial relationships between neighbouring pixels. This allows the model to learn local image structures more effectively than a fully connected representation.

9. Differentiate an autoencoder and a denoising autoencoder.

A conventional autoencoder reconstructs the input image from its latent representation. A denoising autoencoder receives a corrupted image as input but is trained to reconstruct the corresponding clean image.

10. **Why is the clean image used as the target in denoising?**

The clean image is used as the target so that the model learns to remove the introduced corruption while recovering the underlying structure of the original image.

11. **What happens when the noise level is increased?**

Increasing the noise level makes reconstruction more difficult. In the experiment, increasing Gaussian noise from $\sigma = 0.1$ to $\sigma = 0.3$ increased MSE and MAE while decreasing SSIM.

12. **Why are MSE, MAE and SSIM useful for image reconstruction?**

MSE measures the average squared pixel-level error, MAE measures the average absolute pixel-level error, and SSIM measures structural similarity between the original and reconstructed images. Together, they provide complementary measures of reconstruction quality.

13. **What is a Variational Autoencoder?**

A Variational Autoencoder (VAE) is an autoencoder that learns a probability distribution over the latent representation instead of mapping each input to a single deterministic latent vector.

14. **How does a VAE differ from a conventional autoencoder?**

A conventional autoencoder learns a deterministic mapping

$$z = f(x),$$

whereas a VAE learns a distribution characterized by μ and $\log \sigma^2$ and samples a latent vector using the reparameterization trick.

15. **Why does a VAE learn a distribution instead of a single latent vector?**

Learning a distribution provides a structured latent space from which latent vectors can be sampled. This supports both reconstruction and generation of new images.

16. **What is the reparameterization trick?**

The reparameterization trick expresses the latent sample as

$$z = \mu + \sigma \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I).$$

It allows the sampling operation to be used while maintaining a differentiable computation for training.

17. **Why is ϵ sampled from a standard normal distribution?**

The standard normal distribution provides a fixed random source for sampling while the learned parameters μ and σ control the resulting latent distribution.

18. **What is the role of KL divergence in the VAE loss?**

KL divergence regularizes the learned latent distribution by measuring its difference from the chosen prior distribution. It encourages the latent representation to remain close to the prior.

19. **What is the prior distribution used in the VAE?**

The prior distribution used in the experiment is the standard normal distribution:

$$p(z) = \mathcal{N}(0, I).$$

20. **Why is a two-dimensional latent space useful for visualization?**

A two-dimensional latent space allows the encoded samples to be directly plotted on a two-dimensional plane, making the structure and distribution of the learned latent representations easier to examine.

21. **What does latent-space interpolation demonstrate?**

Latent-space interpolation demonstrates how the decoded image changes when moving continuously between two latent vectors. It can be used to examine whether the learned latent representation provides smooth transitions between image representations.

22. **Why can a VAE generate new images?**

A VAE learns a structured latent distribution. New latent vectors can be sampled from the learned prior distribution and passed through the decoder to produce new images.

23. **Why might some VAE-generated images be unclear?**

Some generated images may be unclear because the learned latent representation and decoder do not perfectly model all the variations present in the training images.

24. **What is the difference between reconstruction and generation?**

Reconstruction starts with an existing input image and produces an approximation of that same image. Generation starts from a sampled latent vector and produces a new image through the decoder.

25. **Why should test images remain unseen during training?**

Test images should remain unseen so that the reconstruction metrics measure the model's performance on data that was not used during training.

29. Additional Exercises

1. **Train the convolutional autoencoder with latent dimensions of 4, 8, 16 and 32 and compare reconstruction quality.**

Train the convolutional autoencoder separately using latent dimensions

$$d_z \in \{4, 8, 16, 32\}.$$

For each configuration, calculate MSE, MAE and SSIM and compare the reconstruction quality. The effect of increasing latent capacity on image reconstruction can then be analyzed.

2. **Compare Gaussian noise and salt-and-pepper noise using the same denoising architecture.**

Use the same denoising convolutional autoencoder architecture for both Gaussian and salt-and-pepper corrupted images. Evaluate the reconstructed images using MSE, MAE and SSIM to compare the effect of the two corruption types.

3. **Train the denoising autoencoder using two different noise levels and compare SSIM.**

Train or evaluate the denoising autoencoder using two different noise levels while keeping the architecture and other experimental conditions unchanged. Compare the resulting SSIM values to determine how the amount of corruption affects structural reconstruction quality.

4. **Replace the convolutional decoder with transposed convolutions and compare the reconstructions.**

Replace the upsampling operations in the convolutional decoder with transposed convolution layers. Compare the resulting reconstructions with those obtained using the original decoder architecture using appropriate reconstruction metrics and visual inspection.

5. **Implement a VAE with latent dimension 2 and 8 and compare the latent representations.**

Train VAEs using latent dimensions of 2 and 8. Visualize the two-dimensional latent representation directly and compare the reconstruction performance and representation capacity of the two latent dimensions.

6. **Investigate the effect of increasing the KL-loss contribution.**

Increase the contribution of the KL-divergence term in the VAE loss and observe its effect on the learned latent representation and reconstruction quality.

The VAE objective can be expressed as:

$$\mathcal{L}_{VAE} = \mathcal{L}_{rec} + \beta \mathcal{L}_{KL},$$

where increasing β increases the contribution of the KL-divergence term.

7. **Generate a larger grid of VAE samples and discuss diversity.**

Sample a larger number of latent vectors from

$$z \sim \mathcal{N}(0, I)$$

and pass them through the trained decoder. Display the generated images in a larger grid and examine the variation and diversity of the generated digit samples.

8. **Select two digit regions in the latent space and perform interpolation.**

Select two representative latent points corresponding to different digit regions and perform linear interpolation:

$$z(\alpha) = (1 - \alpha)z_A + \alpha z_B, \quad \alpha \in [0, 1].$$

Decode the interpolated latent vectors and examine the sequence of generated images to study the transition between the two latent regions.

30. Expected Outcome

At the end of the experiment, students should be able to demonstrate the complete image-representation and reconstruction pipeline:

Image $\rightarrow$ Encoder $\rightarrow$ Latent Representation $\rightarrow$ Decoder $\rightarrow$ Reconstructed Image

They should also demonstrate denoising:

Clean Image $\rightarrow$ Noise $\rightarrow$ Denoising Autoencoder $\rightarrow$ Recovered Image

and variational generation:

Image $\rightarrow (\mu, \sigma) \rightarrow z \rightarrow$ Decoder $\rightarrow$ Generated/Reconstructed Image

The final report should contain both quantitative evaluation and visual analysis. Numerical performance values must be obtained from the student's own execution.

31. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Diederik P. Kingma and Max Welling, “Auto-Encoding Variational Bayes,” ICLR, 2014.
3. Pascal Vincent et al., “Stacked Denoising Autoencoders: Learning Useful Representations in a Deep Network with a Local Denoising Criterion,” Journal of Machine Learning Research, 2010.
4. Yann LeCun, Corinna Cortes and Christopher J. C. Burges, “MNIST Handwritten Digit Database.”
5. TensorFlow Documentation: <https://www.tensorflow.org>
6. Keras Documentation: <https://keras.io>
7. scikit-image Documentation: <https://scikit-image.org>

Source Code

The complete implementation is available in the GitHub repository.

GitHub Repository:

<https://github.com/KPR-24110081/Deep-Learning-Lab/tree/main/Ex-7>